Esercizio 3

Introduzione

L’obiettivo di questo esercizio era quello di chiedere ad un'intelligenza artificiale di implementare una hash table con concatenamento che poi verrà usata per calcolare la parola più frequente nel file di testo dato di lunghezza almeno pari ad un valore minimo dato.
L’AI che abbiamo usato è Chat GPT.

Prompt iniziale

Inizialmente, abbiamo dato in pasto all’AI in questione la consegna dell’esercizio e il file hashTable.h.
Abbiamo ottenuto una soluzione che non passava i test su Moodle, però Chat GPT ci consigliava di fare alcuni miglioramenti. Per esempio, uno tra questi miglioramenti era il ridimensionamento dinamico.
Dopodichè abbiamo ottenuto l’implementazione del file hashTable.c

Implementazione

La struttura HashTable è stata implementata utilizzando le liste concatenate per gestire le collisioni.
Le principali funzioni includono =

- **Creazione: hash_table_create** crea una nuova tabella con una capacità iniziale definita.
- **Inserimento: hash_table_put** inserisce o aggiorna una coppia chiave-valore. Se necessario, la tabella viene ridimensionata.
- **Recupero: hash_table_get** restituisce il valore associato a una chiave.
- **Rimozione: hash_table_remove** rimuove una coppia chiave-valore.
- **Gestione memoria: hash_table_free** libera tutte le risorse associate alla tabella.

Elaborazione del file

Infine, gli abbiamo chiesto di implementare il main per cercare la parola più frequente nel file di testo.
Ovviamente la soluzione iniziale non funzionò correttamente ma, in seguito chiedendogli di fare opportune modifiche, siamo arrivati alla soluzione.

Versione iniziale

La prima implementazione presentava limitazioni nella gestione di:

- Caratteri speciali
- File di grandi dimensioni
- Utilizzo della memoria

Versione finale

La funzione find_most_frequent_word è stata ottimizzata per:

- Leggere il file carattere per carattere in modo efficiente
- Costruire le parole rispettando la lunghezza minima richiesta
- Gestire correttamente la memoria durante l'elaborazione

La funzione **find_most_frequent_word** legge il file carattere per carattere, costruendo le parole e aggiungendole alla tabella hash se rispettano la lunghezza minima. Alla fine, scorre le chiavi della tabella per determinare la parola con il conteggio massimo.
Nel main vengono gestiti i parametri di input (nome del file e lunghezza minima), e invoca la funzione di analisi.

Considerazioni finali

Il processo di sviluppo ha evidenziato diversi aspetti:

1) **Efficacia dell'Approccio Iterativo**
- La capacità di ChatGPT di fornire suggerimenti per il miglioramento
- L'importanza del feedback dei test automatici
2) **Qualità del Codice Prodotto**
- Bilanciamento tra efficienza e leggibilità
- Implementazione robusta e ben documentata
- Gestione efficace della memoria
3) **Limitazioni e Miglioramenti**
- Necessità di raffinare i prompt per ottenere risultati ottimali

Il risultato finale è un'implementazione che soddisfa i requisiti, dimostrando l'efficacia dell'utilizzo di ChatGPT nel processo di sviluppo software guidato dai test.